

VARIABLES & DATA TYPES IN JAVA

What is a Variable?

👉 A **variable** is a **named memory location** used to store data.

Imagine it like a container where we define a container with its data type, identifier and the value.

🧠 Example

```
int age = 25;
```

- int → data type
- age → variable name
- 25 → value stored

RULES FOR VARIABLES (IDENTIFIERS)

Rule	Valid Example	Invalid Example	Error
Cannot start with number	int age	int 1age	identifier expected
Cannot use keywords	int marks	int class	keyword error
No special characters	userName	user-name	illegal character
No spaces	totalMarks	total marks	not a statement
Can use _ and \$	_count, \$value	—	—

DATA TYPES IN JAVA

Data is nothing but information and information can be of many types eg. Numbers, characters, words, etc.

So to store an information java has defined some data types to store different types of information.

👉 Java data types are divided into:

◆ A. Primitive Data Types

👉 Store **actual values**.

When we say **actual value** then variable directly holds the **actual value**

Eg `int i=20;`

Stack: `a → 10`

Types of Primitive Data

Type	Size	Example	Description
byte	1 byte	<code>byte b = 10;</code>	small integers
short	2 bytes	<code>short s = 100;</code>	larger than byte
int	4 bytes	<code>int a = 1000;</code>	default integer
long	8 bytes	<code>long l = 10000L;</code>	large integer
float	4 bytes	<code>float f = 1.23f;</code>	decimal
double	8 bytes	<code>double d = 1.23;</code>	default decimal
char	2 bytes	<code>char c = 'A';</code>	single character
boolean	—	<code>boolean b = true;</code>	true/false

⚠️ Prefix Concept (VERY IMPORTANT)

`float f = 1.23f; // ✓ required`

`long l = 10000L; // ✓ required`

✗ Without prefix:

`float f = 1.23; // ✗ ERROR`

👉 Error:

possible lossy conversion from double to float

◆ B. Non-Primitive Data Types

👉 Store **reference (address)**, not actual value

Unlike primitive data types, non-primitive (objects, arrays, String, etc.) → variable holds an **address (reference)**, not the actual data.

Examples:

- String
- Arrays
- Classes
- Objects

Example

```
String name = "Iqbal";
```

Stack: Heap:
name —————▶ "Iqbal"

name does **NOT** store "Iqbal" directly

✓ It stores a **reference (address)** pointing to the actual object in heap

WHAT IS AN OBJECT?

👉 An **object** is an instance of a class

Example

```
class Test {  
}  
Test t = new Test();
```

Explanation

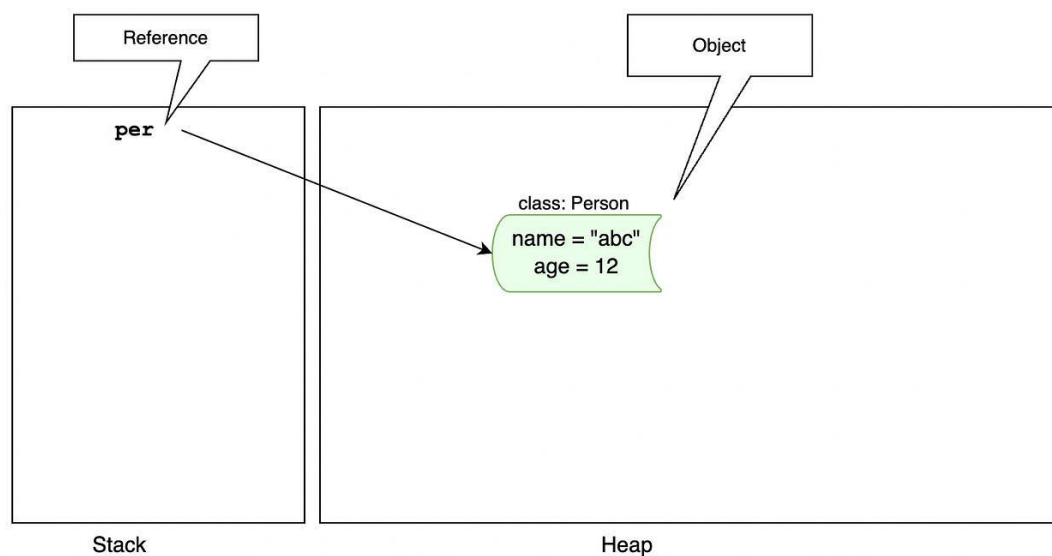
- Test → class
- t → reference variable
- new Test() → object creation

Memory:

- Object stored in heap
- t stores reference (stack area)

```
class Person {  
    private String name;  
    private int age;  
  
    public Person(String, name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
}
```

```
class Test {  
    public static void main(String[] args) {  
        Person per = new Person("abc", 12);  
    }  
}
```



TYPES OF VARIABLES

◆ 1. Local Variables

 Declared inside a method

Example

```
class Demo {  
    void show() {  
        int a = 10; // local variable  
        System.out.println(a);  
    }  
}
```

✓ Rules

- Must be initialized before use
- Cannot use access modifiers (public, private, default, protected);
- Scope limited to method, constructor, blocks.
- Local variables can not be static variable or non-static variables

✗ Error Example

```
int a;  
System.out.println(a); // ✗
```

👉 Error:

variable a might not have been initialized

◆ 2. Instance (Non-Static) Variables

👉 Declared inside class, outside method, without static method.

Example

```
class Demo {  
    int a = 10; // instance variable  
}
```

✓ Rules

- Belongs to object
- Default values assigned
- Accessed using object

Usage

```
Demo d = new Demo();  
System.out.println(d.a);
```

◆ 3. Static Variables

👉 Declared using static keyword

Example

```
class Demo {  
    static int count = 0;  
}
```

✓ Rules

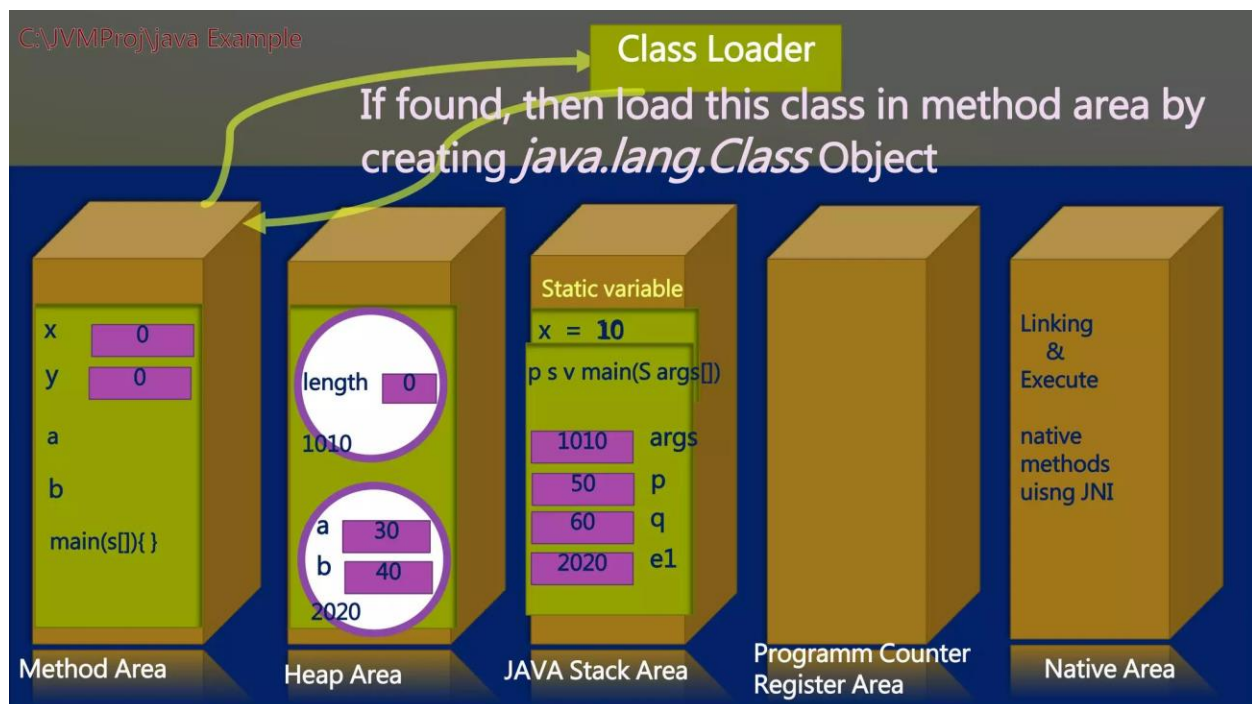
- Shared among all objects
- Accessed using class name
- Only one copy exists

Usage

```
System.out.println(Demo.count);
```

COMPARISON TABLE

Feature	Local	Instance	Static
Location	Inside method	Inside class	Inside class
Scope	Method only	Object	Class
Default value	✗ No	✓ Yes	✓ Yes
Access	Direct	Object	Class



WHAT-IF SCENARIOS

Case 1: Local variable not initialized

```
int a;  
System.out.println(a);
```

✗ Error:

variable a might not have been initialized

Case 2: Access instance variable without object

```
class Demo {  
    int a = 10;  
}
```

```
System.out.println(a); // ❌
```

👉 Error:

non-static variable a cannot be referenced from a static context

Case 3: Access static via object (allowed but not recommended)

```
Demo d = new Demo();  
System.out.println(d.count);
```

✓ Works

⚠️ Bad practice

Case 4: Wrong float assignment

```
float f = 10.5;
```

❌ Error:

possible lossy conversion from double to float

Case 5: Wrong char assignment

```
char c = "A";
```

❌ Error:

incompatible types: String cannot be converted to char

COMPLETE PROGRAM

```
class Demo {  
  
    int instanceVar = 10;  
    static int staticVar = 20;  
  
    void show() {  
        int localVar = 30;  
  
        System.out.println("Local: " + localVar);  
        System.out.println("Instance: " + instanceVar);  
        System.out.println("Static: " + staticVar);  
    }  
  
    public static void main(String[] args) {  
        Demo d = new Demo();  
        d.show();  
    }  
}
```

Output

Local: 30
Instance: 10
Static: 20

PRIMITIVE DATA TYPES

A. Inside NON-STATIC METHOD

Variable Type	Example	Memory Location	Notes
---------------	---------	-----------------	-------

Local Variable	int a = 10;	Stack	Must be initialized
Instance Variable	int x = 20;	Heap (object)	Access via object
Static Variable	static int y = 30;	Method Area	Shared

B. Inside Static Method

Variable Type	Example	Memory Location	Notes
Local Variable	int a = 10;	Stack	Allowed
Instance Variable	int x = 20;	Heap	✗ Cannot access directly
Static Variable	static int y = 30;	Method Area	Access using class

NON-PRIMITIVE DATA TYPES (OBJECTS / STRING)

◆ A. Inside NON-STATIC METHOD

Variable Type	Example	Memory Location	Notes
Local Variable	String s = "Hi";	Stack (ref) + Heap (object)	String pool used
Instance Variable	String name = "Java";	Heap	Part of object
Static Variable	static String company = "ABC";	Method Area + Heap	Shared

◆ B. Inside STATIC METHOD

Variable Type	Example	Memory Location	Notes
Local Variable	String s = "Hi";	Stack + Heap	Allowed
Instance Variable	String name = "Java";	Heap	✗ Cannot access directly
Static Variable	static String company = "ABC";	Method Area	Accessible

Final conclusion

Type	Local Variable	Instance Variable	Static Variable
Primitive	Stack	Heap	Method Area
Non-Primitive	Stack + Heap	Heap	Method Area + Heap

Complete program which show local static and non-static variables.

```
class Test {  
  
    int instanceVar = 10;  
  
    static int staticVar = 20;  
  
    void show() {  
  
        int localVar = 30;  
  
        System.out.println("Local: " + localVar);  
  
        System.out.println("Instance: " + instanceVar);  
  
        System.out.println("Static: " + staticVar);  
  
    }  
}
```

```
public static void main(String[] args) {  
    Test d = new Test();  
    d.show();  
}  
}
```